# LPP GRAPHICAL SOLVER

A Web-Based Two-Variable Linear Programming Solver using the Graphical Method

👉Click here to open the project👈

👉Github👈

---

## 1. Introduction

Linear Programming (LP) is a mathematical method used to find the best possible outcome in a given mathematical model whose requirements are represented by linear relationships. When a problem involves only two decision variables, the Graphical Method provides an intuitive and visually clear approach to find the optimal solution.

This program implements the Graphical Method for solving two-variable LPPs entirely in a web browser using HTML, CSS, and JavaScript — requiring no external software or installation.

## 2. Objective

The program is designed to:

- Accept a user-defined linear objective function Z = cx + dy.
- Accept multiple linear constraints of the form ax + by ≤, ≥, or = RHS.
- Automatically enforce non-negativity constraints (x ≥ 0, y ≥ 0).
- Graphically plot the feasible region, constraint lines, and optimal point.
- Enumerate all corner points and evaluate the objective function at each.
- Identify and highlight the optimal (maximum or minimum) solution.

## 3. Tools and Technologies Used

| Technology | Type | Purpose |
|---|---|---|
| HTML5 | Markup Language | Structure and layout of the web app |
| CSS3 | Styling Language | Visual design, dark theme, animations |
| Vanilla JavaScript | Programming Language | Solver logic, math, canvas rendering |
| HTML5 Canvas API | Browser API | 2D graph plotting and visualization |
| Google Fonts | Web Fonts | Typography: Syne + Space Mono |

## 4. How the Program Works

### 4.1 User Input

The user fills in a form panel on the left side of the interface. They choose to Maximize or Minimize, then enter the coefficients c and d of the objective function. Constraints are added as rows with coefficients a, b, a sign (≤, ≥, =), and a right-hand side value. Constraints can be added or removed dynamically. Non-negativity ($x \geq 0$, $y \geq 0$) is always implicitly enforced.

### 4.2 Finding Corner Points

The solver finds all candidate corner (extreme) points of the feasible region by computing the pairwise intersections of all constraint lines, as well as their intersections with the x-axis and y-axis. The origin (0, 0) is also included. Each candidate point is then tested for feasibility — it must satisfy all constraints simultaneously (with a small numerical tolerance of 1e-9). Points failing any constraint are discarded.

### 4.3 Convex Hull Ordering

The surviving feasible points are deduplicated and ordered angularly around their centroid to form a convex polygon. This ordered set defines the boundary of the feasible region for drawing purposes.

### 4.4 Objective Function Evaluation

The objective value $Z = cx + dy$ is computed at every corner point. The optimal value is the maximum (for maximization) or minimum (for minimization) among these Z values. If multiple points share the same optimal Z, all are highlighted — indicating an infinite number of optimal solutions along a segment.

### 4.5 Graph Rendering (Canvas API)

The graph is drawn on an HTML5 <canvas> element scaled to fit the computed axis range. The following elements are rendered:

- A dark grid background with subtle gridlines.
- Each constraint line in a distinct color, drawn as a dashed line.
- The feasible region as a semi-transparent filled polygon.
- All corner points as circles; the optimal point is highlighted in gold with a ring.
- The iso-profit (or iso-cost) line through the optimal point for visual reference.
- Axis tick labels and a dynamic legend.

## 5. Algorithm Summary

| Step | Description |
|---|---|
| 1 | Read objective function coefficients and optimization type (Max/Min). |
| 2 | Read all constraints; non-negativity is implicit. |
| 3 | Compute pairwise intersections of constraint lines and axis intercepts. |
| 4 | Filter candidate points: retain only those satisfying all constraints. |
| 5 | Deduplicate and order points into a convex polygon (hull). |
| 6 | Evaluate Z at each corner point; find optimal value and corresponding point(s). |
| 7 | Render the graph (region, lines, points) and display results table. |

## 6. Key Features

- **Fully client-side — runs in any modern browser, no server required.**
- Dynamic constraint management — add or remove constraints in real time.
- Supports all constraint types: ≤, ≥, and equality (=).
- Auto-scaling axes based on the feasible region's extent.
- Color-coded constraint lines with a live legend.
- Optimal point ring highlight and iso-profit line overlay.
- Results table showing Z at every corner point, with optimal row marked (★).
- Detects infeasible problems and reports them gracefully.
- Responsive layout that works on desktop and tablet.

## 7. Limitations

- Restricted to two decision variables (x and y) — inherent to the graphical method.
- Does not handle unbounded feasible regions (will produce no corner points).
- Numerical precision relies on a tolerance of 1e-9; near-degenerate problems may behave unexpectedly.
- No support for integer programming (variables are treated as continuous).

## 8. Sample Problem (Default Example)

The program ships with the following default example:

| Component | Expression |
|---|---|
| Objective | Maximize $Z = 5x + 4y$ |
| Constraint 1 | $6x + 4y \leq 24$ |
| Constraint 2 | $x + 2y \leq 6$ |
| Non-negativity | $x \geq 0,\ y \geq 0$ |

Corner points of the feasible region: (0, 0), (4, 0), (3, 1.5), (0, 3). The optimal solution is Z = 21 at the point (3, 1.5).

## 9. Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>LPP Graphical Solver</title>
<style>
  @import
url('https://fonts.googleapis.com/css2?family=Space+Mono:wght@400;700&family=Syne
:wght@400;600;800&display=swap');
```

```
:root {
  --bg: #0a0a0f;
  --surface: #12121a;
  --border: #1e1e2e;
  --accent: #00f5c4;
  --accent2: #ff6b6b;
  --accent3: #ffd93d;
  --text: #e0e0f0;
  --muted: #5a5a7a;
}

* { margin: 0; padding: 0; box-sizing: border-box; }

body {
  background: var(--bg);
  color: var(--text);
  font-family: 'Syne', sans-serif;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 2rem 1rem;
}

/* Grid background */
body::before {
  content: '';
  position: fixed;
  inset: 0;
  background-image:
    linear-gradient(rgba(0,245,196,0.03) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0,245,196,0.03) 1px, transparent 1px);
  background-size: 40px 40px;
  pointer-events: none;
  z-index: 0;
}

.wrapper {
  position: relative;
  z-index: 1;
  width: 100%;
  max-width: 1100px;
}

header {
  text-align: center;
  margin-bottom: 2.5rem;
}

header h1 {
  font-size: clamp(2rem, 5vw, 3.5rem);
  font-weight: 800;
```

```
  letter-spacing: -0.02em;
  background: linear-gradient(135deg, var(--accent), var(--accent3));
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  background-clip: text;
}

header p {
  font-family: 'Space Mono', monospace;
  color: var(--muted);
  font-size: 0.8rem;
  margin-top: 0.5rem;
  letter-spacing: 0.1em;
  text-transform: uppercase;
}

.main-grid {
  display: grid;
  grid-template-columns: 380px 1fr;
  gap: 1.5rem;
  align-items: start;
}

@media (max-width: 768px) {
  .main-grid { grid-template-columns: 1fr; }
}

.panel {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 1.5rem;
  position: relative;
  overflow: hidden;
}

.panel::before {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0;
  height: 2px;
  background: linear-gradient(90deg, transparent, var(--accent), transparent);
}

.panel h2 {
  font-size: 0.75rem;
  letter-spacing: 0.15em;
  text-transform: uppercase;
  color: var(--accent);
  font-family: 'Space Mono', monospace;
  margin-bottom: 1.5rem;
}
```

```
.section-label {
  font-size: 0.7rem;
  font-family: 'Space Mono', monospace;
  color: var(--muted);
  text-transform: uppercase;
  letter-spacing: 0.1em;
  margin-bottom: 0.5rem;
  margin-top: 1rem;
}

.section-label:first-of-type { margin-top: 0; }

/* Radio buttons */
.radio-group {
  display: flex;
  gap: 0.5rem;
  margin-bottom: 1.2rem;
}

.radio-group label {
  flex: 1;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 0.4rem;
  padding: 0.5rem;
  border: 1px solid var(--border);
  border-radius: 8px;
  cursor: pointer;
  font-size: 0.85rem;
  font-family: 'Space Mono', monospace;
  transition: all 0.2s;
  color: var(--muted);
}

.radio-group input { display: none; }

.radio-group input:checked + span,
.radio-group label:has(input:checked) {
  color: var(--bg);
}

.radio-group label:has(input:checked) {
  background: var(--accent);
  border-color: var(--accent);
  color: var(--bg);
}

/* Objective function */
.obj-row {
  display: flex;
  align-items: center;
  gap: 0.5rem;
```

```
  font-family: 'Space Mono', monospace;
  font-size: 0.85rem;
  flex-wrap: wrap;
}

.obj-row span { color: var(--muted); }

input[type="number"] {
  background: var(--bg);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 0.4rem 0.6rem;
  border-radius: 6px;
  font-family: 'Space Mono', monospace;
  font-size: 0.85rem;
  width: 70px;
  outline: none;
  transition: border-color 0.2s;
}

input[type="number"]:focus {
  border-color: var(--accent);
}

/* Constraints */
.constraint-list {
  display: flex;
  flex-direction: column;
  gap: 0.6rem;
  margin-bottom: 1rem;
}

.constraint-row {
  display: flex;
  align-items: center;
  gap: 0.4rem;
  font-family: 'Space Mono', monospace;
  font-size: 0.8rem;
  flex-wrap: wrap;
}

.constraint-row span { color: var(--muted); }

select {
  background: var(--bg);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 0.35rem 0.4rem;
  border-radius: 6px;
  font-family: 'Space Mono', monospace;
  font-size: 0.8rem;
  outline: none;
  cursor: pointer;
```

```
}

select:focus { border-color: var(--accent); }

.btn-add {
  background: transparent;
  border: 1px dashed var(--border);
  color: var(--muted);
  padding: 0.5rem;
  border-radius: 8px;
  cursor: pointer;
  font-family: 'Space Mono', monospace;
  font-size: 0.75rem;
  width: 100%;
  transition: all 0.2s;
  letter-spacing: 0.05em;
}

.btn-add:hover {
  border-color: var(--accent);
  color: var(--accent);
}

.btn-remove {
  background: transparent;
  border: none;
  color: var(--accent2);
  cursor: pointer;
  font-size: 1rem;
  line-height: 1;
  padding: 0 0.2rem;
  opacity: 0.6;
  transition: opacity 0.2s;
}

.btn-remove:hover { opacity: 1; }

.btn-solve {
  width: 100%;
  padding: 0.9rem;
  background: linear-gradient(135deg, var(--accent), #00c49a);
  border: none;
  border-radius: 10px;
  color: var(--bg);
  font-family: 'Syne', sans-serif;
  font-size: 1rem;
  font-weight: 700;
  cursor: pointer;
  letter-spacing: 0.05em;
  transition: all 0.2s;
  margin-top: 1.5rem;
}
```

```
.btn-solve:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 24px rgba(0,245,196,0.3);
}

.btn-solve:active { transform: translateY(0); }

/* Canvas panel */
.canvas-panel {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 1.5rem;
  position: relative;
}

.canvas-panel::before {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0;
  height: 2px;
  background: linear-gradient(90deg, transparent, var(--accent3), transparent);
}

canvas {
  width: 100%;
  height: auto;
  border-radius: 8px;
  background: #080810;
  display: block;
}

/* Results */
.results {
  margin-top: 1.5rem;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 1.5rem;
  position: relative;
}

.results::before {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0;
  height: 2px;
  background: linear-gradient(90deg, transparent, var(--accent2), transparent);
}

.results h2 {
  font-size: 0.75rem;
  letter-spacing: 0.15em;
```

```
  text-transform: uppercase;
  color: var(--accent2);
  font-family: 'Space Mono', monospace;
  margin-bottom: 1rem;
}

.result-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.result-card {
  background: var(--bg);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 1rem;
}

.result-card .label {
  font-family: 'Space Mono', monospace;
  font-size: 0.65rem;
  color: var(--muted);
  text-transform: uppercase;
  letter-spacing: 0.1em;
}

.result-card .value {
  font-size: 1.5rem;
  font-weight: 800;
  color: var(--accent);
  margin-top: 0.2rem;
}

.corner-table {
  width: 100%;
  border-collapse: collapse;
  font-family: 'Space Mono', monospace;
  font-size: 0.75rem;
  margin-top: 1rem;
}

.corner-table th {
  text-align: left;
  color: var(--muted);
  padding: 0.4rem 0.8rem;
  border-bottom: 1px solid var(--border);
  font-weight: 400;
  letter-spacing: 0.05em;
}

.corner-table td {
  padding: 0.5rem 0.8rem;
```

```
    border-bottom: 1px solid rgba(255,255,255,0.03);
    color: var(--text);
  }

  .corner-table tr.optimal td {
    color: var(--accent);
    font-weight: 700;
  }

  .error-msg {
    font-family: 'Space Mono', monospace;
    font-size: 0.8rem;
    color: var(--accent2);
    padding: 1rem;
    border: 1px solid rgba(255,107,107,0.3);
    border-radius: 8px;
    background: rgba(255,107,107,0.05);
  }

  .legend {
    display: flex;
    flex-wrap: wrap;
    gap: 0.8rem;
    margin-top: 1rem;
    font-family: 'Space Mono', monospace;
    font-size: 0.65rem;
  }

  .legend-item {
    display: flex;
    align-items: center;
    gap: 0.4rem;
    color: var(--muted);
  }

  .legend-dot {
    width: 10px;
    height: 10px;
    border-radius: 50%;
  }

  .legend-line {
    width: 20px;
    height: 2px;
  }
</style>
</head>
<body>
<div class="wrapper">
  <header>
    <h1>LPP Graphical Solver</h1>
    <p>Linear Programming Problem — Two-Variable Graphical Method</p>
  </header>
```

```
  <div class="main-grid">
    <!-- Input Panel -->
    <div class="panel">
      <h2>// Problem Setup</h2>

      <div class="section-label">Optimization Type</div>
      <div class="radio-group">
        <label>
          <input type="radio" name="opt" value="max" checked>
          <span>Maximize</span>
        </label>
        <label>
          <input type="radio" name="opt" value="min">
          <span>Minimize</span>
        </label>
      </div>

      <div class="section-label">Objective Function: Z = cx + dy</div>
      <div class="obj-row">
        <span>Z =</span>
        <input type="number" id="obj-c" value="5" step="any">
        <span>x +</span>
        <input type="number" id="obj-d" value="4" step="any">
        <span>y</span>
      </div>

      <div class="section-label">Constraints</div>
      <div class="constraint-list" id="constraints">
        <!-- filled by JS -->
      </div>
      <button class="btn-add" onclick="addConstraint()">+ Add Constraint</button>

      <div style="margin-top:0.8rem; font-family:'Space Mono',monospace; font-
size:0.7rem; color:var(--muted);">
        Non-negativity: x ≥ 0, y ≥ 0 (always applied)
      </div>

      <button class="btn-solve" onclick="solve()">▶ Solve Graphically</button>
    </div>

    <!-- Canvas Panel -->
    <div>
      <div class="canvas-panel">
        <h2>// Feasible Region</h2>
        <canvas id="graph" width="600" height="520"></canvas>
        <div class="legend" id="legend"></div>
      </div>

      <div class="results" id="results" style="display:none;"></div>
    </div>
  </div>
</div>
```

```
<script>
// Default constraints
const defaultConstraints = [
  { a: 6, b: 4, sign: '<=', rhs: 24 },
  { a: 1, b: 2, sign: '<=', rhs: 6 },
];

let constraints = JSON.parse(JSON.stringify(defaultConstraints));

const COLORS =
['#00f5c4','#ff6b6b','#ffd93d','#a78bfa','#60a5fa','#f97316','#34d399'];

function renderConstraints() {
  const list = document.getElementById('constraints');
  list.innerHTML = '';
  constraints.forEach((c, i) => {
    const row = document.createElement('div');
    row.className = 'constraint-row';
    row.innerHTML = `
      <input type="number" value="${c.a}" step="any" style="width:60px"
onchange="constraints[${i}].a=+this.value">
      <span>x</span>
      <select onchange="constraints[${i}].sign=this.value">
        <option ${c.sign=='<='?'selected':''}><=</option>
        <option ${c.sign=='>='?'selected':''}>>= </option>
        <option ${c.sign=='='?'selected':''}>=</option>
      </select>
      <span>... wait</span>
    `;
    // Rebuild properly
    row.innerHTML = `
      <input type="number" value="${c.a}" step="any" style="width:60px"
onchange="constraints[${i}].a=+this.value">
      <span>x +</span>
      <input type="number" value="${c.b}" step="any" style="width:60px"
onchange="constraints[${i}].b=+this.value">
      <span>y</span>
      <select onchange="constraints[${i}].sign=this.value">
        <option value="<=" ${c.sign=='<='?'selected':''}>&le;</option>
        <option value=">=" ${c.sign=='>='?'selected':''}>&ge;</option>
        <option value="=" ${c.sign=='='?'selected':''}>=</option>
      </select>
      <input type="number" value="${c.rhs}" step="any" style="width:60px"
onchange="constraints[${i}].rhs=+this.value">
      ${i>=0 ? `<button class="btn-remove"
onclick="removeConstraint(${i})">X</button>` : ''}
    `;
    list.appendChild(row);
  });
}

function addConstraint() {
```

```
  constraints.push({ a: 1, b: 1, sign: '<=', rhs: 10 });
  renderConstraints();
}

function removeConstraint(i) {
  constraints.splice(i, 1);
  renderConstraints();
}

renderConstraints();

// ---- Math Helpers ----

function lineY(a, b, rhs, x) {
  if (Math.abs(b) < 1e-12) return null;
  return (rhs - a * x) / b;
}

function lineX(a, b, rhs, y) {
  if (Math.abs(a) < 1e-12) return null;
  return (rhs - b * y) / a;
}

function intersect(a1,b1,r1, a2,b2,r2) {
  const det = a1*b2 - a2*b1;
  if (Math.abs(det) < 1e-12) return null;
  const x = (r1*b2 - r2*b1) / det;
  const y = (a1*r2 - a2*r1) / det;
  return { x, y };
}

function satisfies(x, y, c) {
  const lhs = c.a * x + c.b * y;
  if (c.sign === '<=') return lhs <= c.rhs + 1e-9;
  if (c.sign === '>=') return lhs >= c.rhs - 1e-9;
  return Math.abs(lhs - c.rhs) < 1e-9;
}

function isFeasible(x, y) {
  if (x < -1e-9 || y < -1e-9) return false;
  return constraints.every(c => satisfies(x, y, c));
}

function cornerPoints() {
  const pts = [];
  const n = constraints.length;

  // Intersections with axes
  constraints.forEach(c => {
    if (Math.abs(c.a) > 1e-12) {
      const x = c.rhs / c.a;
      if (x >= -1e-9) pts.push({ x: Math.max(0, x), y: 0 });
    }
```

```
    if (Math.abs(c.b) > 1e-12) {
      const y = c.rhs / c.b;
      if (y >= -1e-9) pts.push({ x: 0, y: Math.max(0, y) });
    }
    // intersection with x=0 already done; y=0 too
  });

  // Origin
  pts.push({ x: 0, y: 0 });

  // Pairwise constraint intersections
  for (let i = 0; i < n; i++) {
    for (let j = i+1; j < n; j++) {
      const p = intersect(constraints[i].a, constraints[i].b, constraints[i].rhs,
                          constraints[j].a, constraints[j].b,
constraints[j].rhs);
      if (p) pts.push(p);
    }
    // Each constraint with x=0 and y=0
    const px0 = { x: 0, y: lineY(constraints[i].a, constraints[i].b,
constraints[i].rhs, 0) };
    if (px0.y !== null && isFinite(px0.y)) pts.push(px0);
    const py0 = { y: 0, x: lineX(constraints[i].a, constraints[i].b,
constraints[i].rhs, 0) };
    if (py0.x !== null && isFinite(py0.x)) pts.push(py0);
  }

  // Intersections between axes (trivial)
  pts.push({ x: 0, y: 0 });

  return pts.filter(p => isFinite(p.x) && isFinite(p.y) && p.x >= -1e-9 && p.y >=
-1e-9)
            .map(p => ({ x: Math.max(0, p.x), y: Math.max(0, p.y) }))
            .filter(p => isFeasible(p.x, p.y));
}

function dedupe(pts) {
  const out = [];
  pts.forEach(p => {
    if (!out.some(q => Math.abs(q.x-p.x)<1e-7 && Math.abs(q.y-p.y)<1e-7))
out.push(p);
  });
  return out;
}

function convexHull(pts) {
  if (pts.length < 2) return pts;
  const cx = pts.reduce((s,p)=>s+p.x,0)/pts.length;
  const cy = pts.reduce((s,p)=>s+p.y,0)/pts.length;
  return [...pts].sort((a,b)=>Math.atan2(a.y-cy,a.x-cx)-Math.atan2(b.y-cy,b.x-
cx));
}
```

```
// ---- Drawing ----
function solve() {
  const c = +document.getElementById('obj-c').value;
  const d = +document.getElementById('obj-d').value;
  const optType = document.querySelector('input[name="opt"]:checked').value;

  const cp = dedupe(cornerPoints());

  const canvas = document.getElementById('graph');
  const ctx = canvas.getContext('2d');
  const W = canvas.width, H = canvas.height;
  const PAD = { left: 55, right: 30, top: 30, bottom: 50 };
  const PW = W - PAD.left - PAD.right;
  const PH = H - PAD.top - PAD.bottom;

  // Determine axis range
  let maxX = 0, maxY = 0;
  cp.forEach(p => { maxX = Math.max(maxX, p.x); maxY = Math.max(maxY, p.y); });
  // Also include constraint axis intercepts for display
  constraints.forEach(c => {
    if (Math.abs(c.a) > 1e-12) maxX = Math.max(maxX, Math.abs(c.rhs / c.a));
    if (Math.abs(c.b) > 1e-12) maxY = Math.max(maxY, Math.abs(c.rhs / c.b));
  });
  maxX = Math.max(maxX * 1.25, 10);
  maxY = Math.max(maxY * 1.25, 10);

  const toCanvasX = x => PAD.left + (x / maxX) * PW;
  const toCanvasY = y => PAD.top + PH - (y / maxY) * PH;

  // Clear
  ctx.clearRect(0, 0, W, H);
  ctx.fillStyle = '#080810';
  ctx.fillRect(0, 0, W, H);

  // Grid
  ctx.strokeStyle = 'rgba(255,255,255,0.04)';
  ctx.lineWidth = 1;
  const nGrid = 8;
  for (let i = 0; i <= nGrid; i++) {
    const gx = PAD.left + (i/nGrid)*PW;
    ctx.beginPath(); ctx.moveTo(gx, PAD.top); ctx.lineTo(gx, PAD.top+PH);
ctx.stroke();
    const gy = PAD.top + (i/nGrid)*PH;
    ctx.beginPath(); ctx.moveTo(PAD.left, gy); ctx.lineTo(PAD.left+PW, gy);
ctx.stroke();
  }

  // Draw feasible region
  const hull = convexHull(cp);
  if (hull.length >= 3) {
    ctx.beginPath();
    ctx.moveTo(toCanvasX(hull[0].x), toCanvasY(hull[0].y));
    hull.forEach(p => ctx.lineTo(toCanvasX(p.x), toCanvasY(p.y)));
```

```
    ctx.closePath();
    ctx.fillStyle = 'rgba(0,245,196,0.08)';
    ctx.fill();
    ctx.strokeStyle = 'rgba(0,245,196,0.3)';
    ctx.lineWidth = 1.5;
    ctx.stroke();
  }

  // Draw constraint lines
  constraints.forEach((con, i) => {
    const col = COLORS[i % COLORS.length];
    ctx.strokeStyle = col;
    ctx.lineWidth = 1.8;
    ctx.setLineDash([6,3]);
    ctx.beginPath();
    let started = false;
    for (let px = 0; px <= PW; px += 2) {
      const wx = (px / PW) * maxX;
      const wy = lineY(con.a, con.b, con.rhs, wx);
      if (wy === null || !isFinite(wy)) continue;
      if (wy < -0.5 || wy > maxY * 1.1) {
        if (started) { ctx.stroke(); started = false; ctx.beginPath(); }
        continue;
      }
      const cx2 = toCanvasX(wx), cy2 = toCanvasY(wy);
      if (!started) { ctx.moveTo(cx2, cy2); started = true; }
      else ctx.lineTo(cx2, cy2);
    }
    // vertical line if b=0
    if (Math.abs(con.b) < 1e-12 && Math.abs(con.a) > 1e-12) {
      const vx = con.rhs / con.a;
      if (vx >= 0 && vx <= maxX) {
        ctx.moveTo(toCanvasX(vx), toCanvasY(0));
        ctx.lineTo(toCanvasX(vx), toCanvasY(maxY));
      }
    }
    ctx.stroke();
    ctx.setLineDash([]);
  });

  // Axes
  ctx.strokeStyle = 'rgba(255,255,255,0.5)';
  ctx.lineWidth = 1.5;
  ctx.beginPath();
  ctx.moveTo(PAD.left, PAD.top); ctx.lineTo(PAD.left, PAD.top+PH+8);
ctx.stroke();
  ctx.beginPath();
  ctx.moveTo(PAD.left-8, PAD.top+PH); ctx.lineTo(PAD.left+PW, PAD.top+PH);
ctx.stroke();

  // Axis labels
  ctx.fillStyle = 'rgba(255,255,255,0.4)';
  ctx.font = '11px Space Mono, monospace';
```

```
  ctx.textAlign = 'center';
  for (let i = 0; i <= nGrid; i++) {
    const val = (i/nGrid)*maxX;
    if (val > 0) {
      ctx.fillText(Math.round(val*10)/10, toCanvasX(val), PAD.top+PH+16);
    }
    const valy = (i/nGrid)*maxY;
    if (valy > 0) {
      ctx.textAlign = 'right';
      ctx.fillText(Math.round(valy*10)/10, PAD.left-6, toCanvasY(valy)+4);
      ctx.textAlign = 'center';
    }
  }

  // Axis name
  ctx.fillStyle = 'rgba(255,255,255,0.5)';
  ctx.font = 'bold 12px Space Mono, monospace';
  ctx.fillText('x', PAD.left+PW+10, PAD.top+PH+4);
  ctx.fillText('y', PAD.left-10, PAD.top-8);

  // Corner points
  const objVals = cp.map(p => ({ ...p, z: c*p.x + d*p.y }));
  const optVal = optType === 'max'
    ? Math.max(...objVals.map(o=>o.z))
    : Math.min(...objVals.map(o=>o.z));
  const optPts = objVals.filter(o => Math.abs(o.z - optVal) < 1e-7);

  objVals.forEach(p => {
    const isOpt = Math.abs(p.z - optVal) < 1e-7;
    const px = toCanvasX(p.x), py = toCanvasY(p.y);
    ctx.beginPath();
    ctx.arc(px, py, isOpt ? 7 : 5, 0, 2*Math.PI);
    ctx.fillStyle = isOpt ? '#ffd93d' : 'rgba(255,255,255,0.6)';
    ctx.fill();
    if (isOpt) {
      ctx.strokeStyle = '#ffd93d';
      ctx.lineWidth = 2;
      ctx.beginPath();
      ctx.arc(px, py, 12, 0, 2*Math.PI);
      ctx.stroke();
    }
    // Label
    ctx.fillStyle = isOpt ? '#ffd93d' : 'rgba(255,255,255,0.7)';
    ctx.font = `${isOpt?'bold ':''}10px Space Mono, monospace`;
    ctx.textAlign = 'left';
    ctx.fillText(`(${Math.round(p.x*100)/100}, ${Math.round(p.y*100)/100})`,
px+10, py-6);
  });

  // Draw optimal objective line (iso-profit line through optimal point)
  if (optPts.length > 0) {
    const op = optPts[0];
    ctx.save();
```

```
    ctx.strokeStyle = 'rgba(255,217,61,0.5)';
    ctx.lineWidth = 1.5;
    ctx.setLineDash([4,4]);
    // Line: cx + dy = optVal
    ctx.beginPath();
    let s2 = false;
    for (let px2 = 0; px2 <= PW; px2 += 2) {
      const wx = (px2/PW)*maxX;
      const wy = lineY(c, d, optVal, wx);
      if (wy === null || !isFinite(wy) || wy < -0.5 || wy > maxY*1.1) {
        if (s2) { ctx.stroke(); s2=false; ctx.beginPath(); }
        continue;
      }
      const ex = toCanvasX(wx), ey = toCanvasY(wy);
      if (!s2) { ctx.moveTo(ex,ey); s2=true; }
      else ctx.lineTo(ex,ey);
    }
    ctx.stroke();
    ctx.setLineDash([]);
    ctx.restore();
  }

  // Legend
  const legendEl = document.getElementById('legend');
  legendEl.innerHTML = '';
  constraints.forEach((con, i) => {
    const col = COLORS[i % COLORS.length];
    const sign = con.sign === '<=' ? '≤' : con.sign === '>=' ? '≥' : '=';
    const div = document.createElement('div');
    div.className = 'legend-item';
    div.innerHTML = `<div class="legend-line" style="background:${col}"></div>
${con.a}x + ${con.b}y ${sign} ${con.rhs}`;
    legendEl.appendChild(div);
  });
  // Optimal point legend
  const div2 = document.createElement('div');
  div2.className = 'legend-item';
  div2.innerHTML = `<div class="legend-dot" style="background:#ffd93d"></div>
Optimal Point`;
  legendEl.appendChild(div2);
  const div3 = document.createElement('div');
  div3.className = 'legend-item';
  div3.innerHTML = `<div class="legend-line"
style="background:rgba(0,245,196,0.5); height:8px; opacity:0.5; border-
radius:4px; width:16px"></div> Feasible Region`;
  legendEl.appendChild(div3);

  // Results panel
  const resultsEl = document.getElementById('results');
  if (cp.length === 0) {
    resultsEl.style.display = 'block';
    resultsEl.innerHTML = `<h2>// Solution</h2><div class="error-msg">⚠ No
feasible region found. The constraints may be infeasible.</div>`;
```

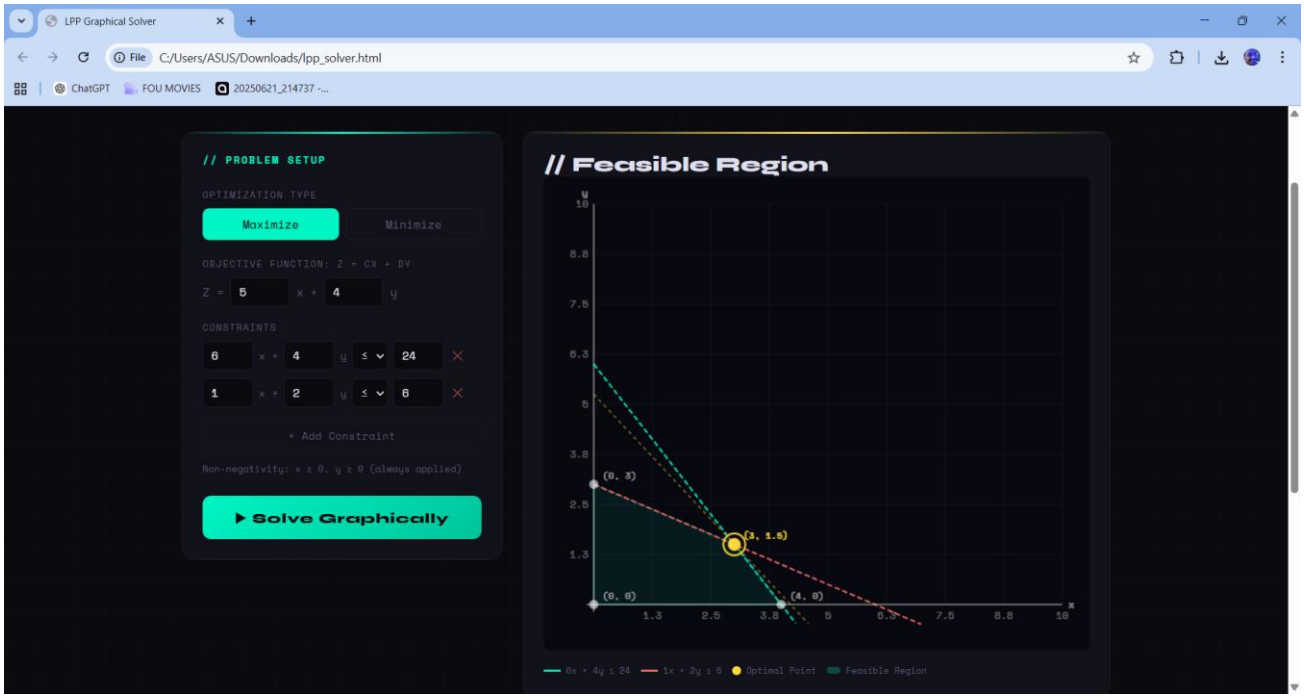
```
    return;
  }

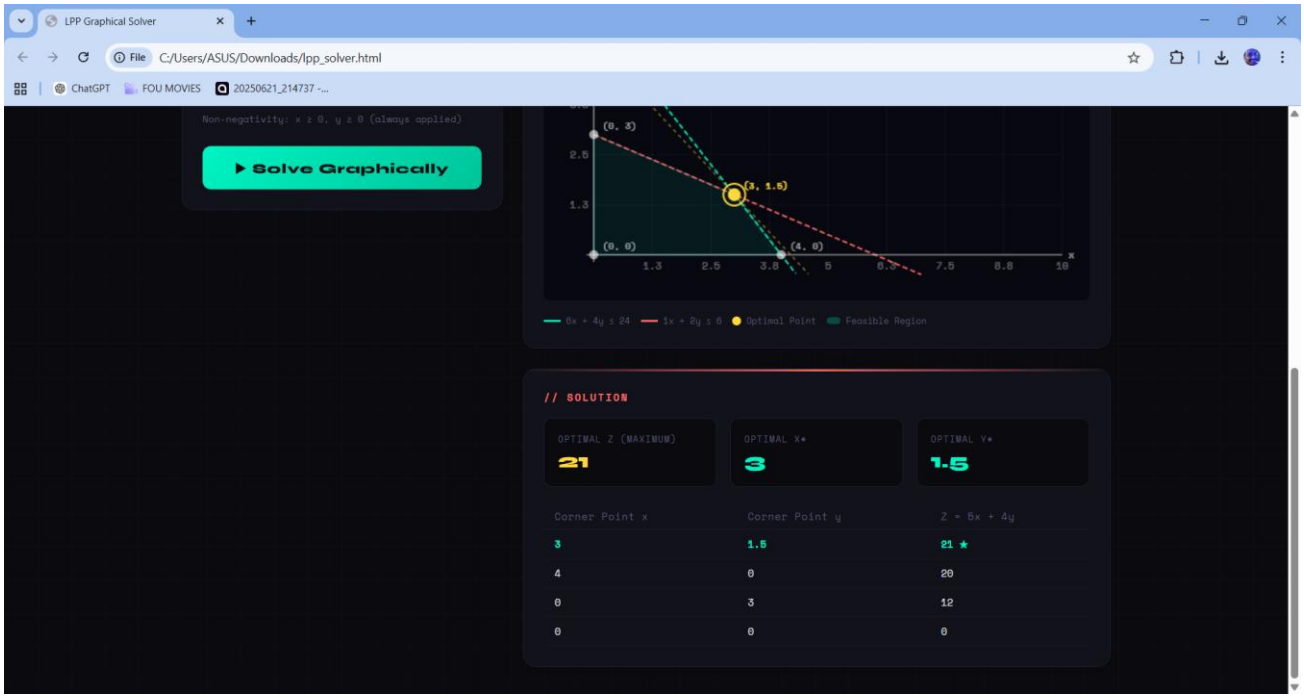
  let tableRows = '';
  objVals.sort((a,b) => optType==='max' ? b.z-a.z : a.z-b.z).forEach(p => {
    const isOpt = Math.abs(p.z - optVal) < 1e-7;
    tableRows += `<tr class="${isOpt?'optimal':''}">
      <td>${Math.round(p.x*10000)/10000}</td>
      <td>${Math.round(p.y*10000)/10000}</td>
      <td>${Math.round(p.z*10000)/10000}${isOpt?' ★':''}</td>
    </tr>`;
  });

  resultsEl.style.display = 'block';
  resultsEl.innerHTML = `
    <h2>// Solution</h2>
    <div class="result-grid">
      <div class="result-card">
        <div class="label">Optimal Z (${optType === 'max' ? 'Maximum' :
'Minimum'})</div>
        <div class="value" style="color:var(--
accent3)">${Math.round(optVal*10000)/10000}</div>
      </div>
      ${optPts.map(p => `
      <div class="result-card">
        <div class="label">Optimal x*</div>
        <div class="value">${Math.round(p.x*10000)/10000}</div>
      </div>
      <div class="result-card">
        <div class="label">Optimal y*</div>
        <div class="value">${Math.round(p.y*10000)/10000}</div>
      </div>`).join('')}
    </div>
    <table class="corner-table" style="margin-top:1.2rem">
      <thead>
        <tr>
          <th>Corner Point x</th>
          <th>Corner Point y</th>
          <th>Z = ${c}x + ${d}y</th>
        </tr>
      </thead>
      <tbody>${tableRows}</tbody>
    </table>
  `;
}

// Auto-solve on load
window.onload = () => solve();
</script>
</body>
</html>
```

## 10. Screenshot




## 11. Conclusion

This program successfully implements the Graphical Method for solving two-variable Linear Programming Problems in a fully interactive web interface. By combining real-time user input, mathematical intersection algorithms, and HTML5 Canvas rendering, it delivers a clear, educational, and functional solver. The dark-themed, minimalist design with color-coded annotations makes it easy to understand the solution process — making it useful as both a learning tool and a quick computational aid.